La Faille

Traverser la Faille

Nyx estime que vous pouvez maintenant traverser la zone la plus instable de JS Nexus.

Les opérations doivent être exécutées dans un ordre lisible, même lorsqu'elles retournent des promesses.

> « Attendre n'est pas bloquer. Il faut simplement savoir quand reprendre. »

Vous allez utiliser :

- `async` ;
- `await` ;
- `try / catch` avec des opérations asynchrones.

Instructions

Complétez les fonctions du fichier `faille.js`.

Les fonctions reçoivent parfois en paramètre un **loader**.
Un loader est une fonction qui retourne une `Promise`.

Exemple :

```
const loader = () => Promise.resolve("Fragment A")
```

1. Attendre une valeur

Implémentez :

```
loadFragment(loader)
```

La fonction doit être déclarée `async`.

Appelez `loader()` puis utilisez `await` pour récupérer sa valeur et la retourner.

2. Construire un message après chargement

Implémentez :

```
fragmentStatus(loader)
```

Attendez la valeur retournée par `loader()` puis retournez :

```
Fragment reçu : <valeur>
```

3. Gérer une erreur asynchrone

Implémentez :

```
safeLoad(loader)
```

Utilisez `try / catch`.

- si le loader réussit, retournez sa valeur ;
- s'il échoue, retournez `"Fragment indisponible"`.

4. Charger deux fragments dans l'ordre

Implémentez :

```
loadSequence(firstLoader, secondLoader)
```

Utilisez `await` pour charger d'abord le premier fragment, puis le second.

Retournez :

```
[first, second]
```

Exemple :

```
await loadSequence(
  () => Promise.resolve("A"),
  () => Promise.resolve("B")
)
// => ["A", "B"]
```

5. Vérifier un portail distant

Implémentez :

```
openPortal(loader)
```

Le loader retourne une promesse contenant un booléen.

- si la valeur obtenue est `true`, retournez `"Portail ouvert"` ;
- sinon retournez `"Portail fermé"` ;
- si le loader rejette la promesse, retournez `"Erreur temporelle"`.

Utilisez `async`, `await` et `try / catch`.

À rechercher si nécessaire

- `JavaScript async function`
- `JavaScript await`
- `JavaScript async try catch`

▶ Aides